VoiceForge

Technical Documentation

A voice- and text-driven application factory: family and friends describe an app in plain language; VoiceForge plans it with AI agents, generates a Next.js application, tests it in an isolated environment, and deploys it to the web — with explicit human approval gates before any build and before any production release.

Document version 1.0 · July 2026 · covers Stages 0–6 (complete)

Contents

1. System Overview

VoiceForge is a multi-tenant web application (a Next.js 15 monolith) that orchestrates AI agents and third-party platform APIs to turn conversational app descriptions into deployed, tested web applications. It consists of seven cooperating subsystems, mirroring the original specification: (A) the user-facing web/PWA app, (B) the voice/session layer, (C) the agent orchestrator, (D) the guarded tool layer, (E) the code sandbox/test environment, (F) the GitHub/Vercel deployment layer, and (G) the app registry database.

Two invariants govern every flow: **no code is generated without an explicit user approval** recorded in the database, and **no production deployment happens without a second explicit approval** (the Publish action). All privileged credentials (OpenAI, GitHub, Vercel, database) exist only server-side; browsers receive at most a short-lived ephemeral voice token.

Hosting topology

VoiceForge runs identically in two modes. **Local mode:** the owner runs the server on a Mac (dev or production Node server); generated-app builds execute as local child processes in a temp workspace. **Hosted mode:** the same codebase deployed on Vercel; builds execute inside Vercel Sandbox microVMs, background work is registered via waitUntil(), and long deployment waits are finalized by the status-polling heartbeat rather than blocking function time. Generated apps are always hosted on Vercel (one project per app) regardless of mode.

2. Software Stack

Layer	Technology	Version	Notes
Framework	Next.js (App Router)	^15.5.18	React 19.2; TypeScript 5.9 strict
Styling	Tailwind CSS	^4.3	CSS-first config via @theme tokens
Auth	Clerk (@clerk/nextjs)	^7.5.16	Email magic link; Restricted (invite-only) sign-up
Database	Neon Postgres (serverless)	@neondatabase/serverless ^1.1	HTTP driver
ORM	Drizzle ORM / drizzle-kit	^0.45.2 / ^0.31.10	Schema-as-code; db:push migrations
AI orchestration	OpenAI Agents SDK (TS)	@openai/agents ^0.13.1	Agents, tools, realtime
Voice	OpenAI Realtime over WebRTC	gpt-realtime-2.1-mini	Ephemeral client secrets
Planning model	OPENAI_PLANNER_MODEL	default gpt-5.4-mini	Env-swappable
Coding model	OPENAI_CODER_MODEL	default gpt-5.4	Code, change, debug agents
GitHub API	@octokit/rest	^22.0.1	Fine-grained PAT auth
Deploy API	Vercel REST API	v9/v10/v11/v13	Raw fetch, bearer token
Cloud builds	@vercel/sandbox	^2.5.0	node24 microVMs, hosted mode only
Background	@vercel/functions	^3.7.5	waitUntil registration
Validation	Zod	^4.4.3	Specs, request bodies, agent tool params
Generated apps	Next.js 15 + Tailwind 4 + Vitest 4	template-pinned	Client-side only; localStorage
Gen-app testing	Vitest, @testing-library/react, jsdom	^4.1.8 / ^16.3.2	jest-dom matchers preloaded

3. Technical Architecture

Subsystem map

Subsystem	Implementation	Responsibility
A. User-facing app	src/app/** pages + components	Dashboard, planning chats, build status, approvals, publish, rollback, delete, admin
B. Voice/session layer	api/realtime/session, VoiceChat.tsx	Mints ephemeral keys; WebRTC session in browser; transcripts persisted
C. Agent orchestrator	src/lib/agents/*	Planner, Change Planner, Code, Change, Debug agents (OpenAI Agents SDK)
D. Tool layer	src/lib/{github,vercel}.ts + guarded tools	Every external mutation is a named, audited function; agents cannot call platforms directly
E. Sandbox/test env	src/lib/build/runner.ts	Local child processes or Vercel Sandbox microVMs; install→typecheck→lint→test→build
F. Deployment layer	src/lib/build/{pipeline,publish,finalize}.ts	Branch commits, preview deployments, smoke tests, production promote, rollback
G. App registry	src/db/schema.ts (10 tables)	Users, apps, specs, approvals, builds, deployments, tests, changes, audit

Trust boundaries

Browser ↔ server: Clerk session cookies authenticate every API route; ownership checks scope all queries to the signed-in user (admins may read all apps). The only credential ever sent to a browser is a ≤15-minute OpenAI Realtime client secret (ek_*).
Server ↔ platforms: OPENAI_API_KEY, GITHUB_TOKEN (fine-grained PAT), VERCEL_TOKEN, and DATABASE_URL live only in server env vars.
Agents ↔ system: agents mutate nothing directly. Code-writing agents emit files through a write_file tool that enforces a path policy; platform actions happen in deterministic pipeline code after agent output is validated with Zod.

Generated-app isolation

Every generated app starts from a locked template (templates/nextjs-base). Agents may only write .ts/.tsx files under src/app/, src/components/, and src/lib/; package.json, all configs, globals.css, and the baseline test are immutable, so the dependency set is pinned and npm install always runs with --ignore-scripts. Generated apps are client-side only (localStorage persistence), contain no secrets, and each lives in its own private GitHub repo and own Vercel project (repo/project name: voiceforge-<slug>). A vercel.json with git.deploymentEnabled=false prevents duplicate auto-builds; VoiceForge alone decides when deployments happen.

4. Repository Layout and Module Reference

Path	Description
src/app/page.tsx	Landing page; redirects signed-in users to /dashboard
src/app/sign-in/[[...sign-in]]	Clerk hosted sign-in component route
src/app/dashboard/layout.tsx	Nav shell (My Apps / Create / Change / Admin*), Clerk UserButton (*admin role only)
src/app/dashboard/page.tsx	My Apps grid with live status chips
src/app/dashboard/create/page.tsx	Text planning chat (PlannerChat)
src/app/dashboard/change/page.tsx	Picker of built apps eligible for change
src/app/dashboard/change/[appId]	Change conversation for one app
src/app/dashboard/voice/page.tsx	Voice planning (create or ?app= change mode)
src/app/dashboard/apps/[appId]	Build status, preview/publish, version history, delete
src/app/dashboard/admin/page.tsx	Admin: stats, all apps, recent builds, audit log
src/app/api/chat	POST — planning turn (create & change modes)
src/app/api/approvals/[approvalId]	POST — approve/reject; queues build on approval
src/app/api/apps/[appId]	DELETE — remove app from Vercel, GitHub, VoiceForge
src/app/api/apps/[appId]/status	GET — poll: run+tests+logs; heartbeat (finalize, reap)
src/app/api/apps/[appId]/rebuild	POST — re-run pipeline from latest approved spec
src/app/api/apps/[appId]/publish	POST — production approval; merge + prod deploy
src/app/api/apps/[appId]/rollback	POST — instant rollback/promote to prior deployment
src/app/api/realtime/session	POST — mint ephemeral voice key + conversation row
src/app/api/voice/propose	POST — persist spec proposed by voice tool call
src/app/api/voice/transcript	POST — save transcript when session ends
src/lib/agents/planner.ts	Planner + Change Planner agents (text)
src/lib/agents/coder.ts	Code, Change and Debug agents; write_file tool; shared rules
src/lib/build/pipeline.ts	Build state machine (generate→test→debug→commit→deploy)
src/lib/build/publish.ts	Merge build branch to main; create production deployment
src/lib/build/finalize.ts	Poll-driven completion of pending deployments + smoke test
src/lib/build/runner.ts	Runner abstraction: local child-process / Vercel Sandbox
src/lib/build/template.ts	Template loader, placeholder substitution, path policy
src/lib/github.ts	Repo create/delete, branch, snapshot commit, tree fetch, merge
src/lib/vercel.ts	Project ensure/delete, deployments, restore, smoke test
src/lib/proposals.ts	Shared proposal persistence (text + voice)
src/lib/spec.ts	Zod appSpecSchema + changeProposalSchema
src/lib/users.ts / audit.ts / quota.ts / slug.ts / background.ts	User sync from Clerk; audit writer; quotas + stale-run reaper; slug uniqueness; waitUntil wrapper
src/components/*	PlannerChat, VoiceChat, BuildStatus, VersionHistory, DeleteAppButton
src/middleware.ts	clerkMiddleware protecting /dashboard and /api
templates/nextjs-base/**	Locked generated-app template (14 files)

5. Database Schema

Postgres (Neon), managed by Drizzle ORM. All primary keys are UUIDs (gen_random_uuid()); all timestamps are timestamptz. Schema is pushed with drizzle-kit (npm run db:push).

Enumerations

Enum	Values
user_role	admin, user
app_status	draft, spec_approved, building, testing, deployed, failed, archived
conversation_channel	text, voice
approval_type	build, change, deploy_production
approval_status	pending, approved, rejected
build_run_status	queued, generating, testing, debugging, deploying, awaiting_user_test, complete, failed, needs_input
deployment_environment	preview, production
deployment_status	queued, building, ready, error, canceled
test_suite	typecheck, lint, unit, component, e2e, accessibility, security, build, smoke
test_status	passed, failed, skipped
change_request_status	intake, clarifying, awaiting_approval, building, complete, rejected, failed

users

One row per Clerk account, created on first sign-in.

Column	Type	Notes
id	uuid PK	
clerk_user_id	text UNIQUE	Clerk subject id (users_clerk_user_id_idx)
email	text	Primary email at first sign-in
display_name	text NULL	Friendly name
role	user_role	'admin' if email is in ADMIN_EMAILS at creation; else 'user'
monthly_build_limit	integer	Default 10; enforced per calendar month; admins exempt
created_at / updated_at	timestamptz	

apps

One row per generated app.

Column	Type	Notes
id	uuid PK	
owner_id	uuid FK users	Owner; unique (owner_id, slug)
name / slug	text	Display name; url-safe slug (repo/project = voiceforge-)
description	text NULL	Spec purpose
status	app_status	Lifecycle chip on dashboard
github_repo_url	text NULL	Set after first successful build
vercel_project_id	text NULL	Set when Vercel project is created

Column	Type	Notes
preview_url / production_url	text NULL	Latest preview; live URL
created_at / updated_at	timestamptz	

conversations

Planning session transcripts (text and voice).

Column	Type	Notes
id	uuid PK	
app_id	uuid FK NULL	Set when a proposal creates/targets an app; nulled on app delete (history kept)
user_id	uuid FK users	
channel	conversation_channel	text | voice
transcript	jsonb	Text mode: Agents-SDK history items. Voice mode: [{role, text}] lines
created_at / updated_at	timestamptz	

requirements

Versioned structured specs (the contract for a build).

Column	Type	Notes
id	uuid PK	
app_id	uuid FK apps	unique (app_id, version)
version	integer	1 = original; increments on every proposal/change
spec	jsonb	AppSpec (see §6): name, purpose, users, screens, features, dataToStore, needsLogin, sharingModel, aiFeatures, testPlan, deploymentNotes
plain_summary	text NULL	Plain-English summary shown at approval
created_by	uuid FK users NULL	
created_at	timestamptz	

approvals

Human decision records; nothing builds or ships without one.

Column	Type	Notes
id	uuid PK	
app_id	uuid FK apps	
requirement_id	uuid FK NULL	Spec version being approved
user_id	uuid FK users	Decider
type	approval_type	build (new app), change (modification), deploy_production (publish)
status	approval_status	pending → approved | rejected; superseded pendings auto-rejected
decided_at	timestamptz NULL	
created_at	timestamptz	

build_runs

Durable state machine for every build job.

Column	Type	Notes
id	uuid PK	
app_id / requirement_id / approval_id	uuid FKs	Traceability of what/why
status	build_run_status	See §8 state machine
branch	text NULL	build-
commit_sha	text NULL	Snapshot commit
logs	jsonb	Append-only [{ts, message}] shown live in UI
error_message	text NULL	Terminal failure reason
started_at / finished_at / created_at	timestamptz	created_at drives quota + stale reaping (25 min)

deployments

Every Vercel deployment created by VoiceForge.

Column	Type	Notes
id	uuid PK	
app_id / build_run_id	uuid FKs	
environment	deployment_environment	preview | production
vercel_deployment_id	text NULL	dpl_* id used for polling, rollback
url	text NULL	https URL once ready
status	deployment_status	building → ready | error (finalizer-driven)
created_at	timestamptz	Ordering = version history

test_results

Outcome of each pipeline check, every attempt.

Column	Type	Notes
id	uuid PK	
build_run_id	uuid FK	
suite	test_suite	install/build map to 'build'; unit tests to 'unit'; smoke to 'smoke'
status	test_status	
summary	text NULL	e.g. 'typecheck (3s)'
details	jsonb NULL	{output}: 8 KB tail of the step output
created_at	timestamptz	

change_requests

Lifecycle of 'change my app' requests.

Column	Type	Notes
id	uuid PK	
app_id / user_id	uuid FKs	
description	text	Plain-language changeSummary from the planner
status	change_request_status	awaiting_approval → building → complete | rejected | failed
requirement_id	uuid FK NULL	The new spec version implementing the change
created_at / updated_at	timestamptz	

audit_logs

Append-only record of every sensitive action.

Column	Type	Notes
id	uuid PK	
user_id / app_id / build_run_id	uuid FKs NULL	Nulled (not deleted) when parents are removed
action	text	e.g. spec.proposed, approval.decided, github.repoCreated, github.commit, vercel.projectCreated, vercel.deploymentCreated, vercel.deploymentRestored, build.started/completed/failed, app.published, app.rolledBack, app.deleted, voice.sessionStarted
payload	jsonb NULL	Sanitized arguments/results; never secrets
created_at	timestamptz	Indexed for admin timeline

6. AI Agents and Models

All agents run server-side through the OpenAI Agents SDK for TypeScript (@openai/agents). Models are env-configurable without code changes. Newer reasoning models emit user-facing text in 'commentary'-phase messages with an empty final_answer; VoiceForge therefore extracts replies by concatenating all non-empty assistant output_text items rather than relying on finalOutput.

Agent	Model (default)	Tools	Role
Planner	gpt-5.4-mini	propose_spec (Zod appSpecSchema)	Intake + Clarifier + Product Spec for new apps: ≤2 questions per turn, ~3–5 rounds, suggests a name, records the structured spec exactly once
Change Planner	gpt-5.4-mini	propose_change (spec + changeSummary)	Same, seeded with the app's current spec; returns the complete updated spec
Voice Planner(s)	gpt-realtime-2.1-mini	propose_spec / propose_change (browser-side tools)	Speech-to-speech versions of the above over WebRTC; tool handlers POST to /api/voice/propose
Code Agent	gpt-5.4	write_file (path-policy enforced)	Generates the full app from the spec on top of the locked template; writes tests
Change Agent	gpt-5.4	write_file	Receives current src files from GitHub + updated spec + changeSummary; rewrites only what must change; preserves localStorage data shapes
Debug Agent	gpt-5.4	write_file	Receives failing step output, all src files, and notes from earlier debug rounds this build; must escalate strategy if the same test was already 'fixed'

Prompt-engineering rules encoded in the agents

The shared coder/debugger rule set encodes every failure class discovered during verification: server prerendering (never touch window/localStorage outside useEffect); App Router purity (no pages/, 404.tsx, _document); deterministic tests (no fake timers, no setTimeout-driven logic, fireEvent not user-event, role/label queries not text matching); no references to static assets the agent cannot create (Web Audio synthesis for sound, inline SVG for graphics, localStorage data-URLs for user images); pinned dependencies (imports limited to react, next, and self-written files). The Debug Agent additionally carries known failure signatures (e.g. the misleading <Html> prerender error) and an escalation rule: after a failed fix of the same test, repair the component or simplify the test rather than re-massaging queries.

AI usage costs (approximate)

Activity	Model	Typical cost
Text planning conversation	gpt-5.4-mini	$0.01–0.03
Voice planning conversation	gpt-realtime-2.1-mini + gpt-4o-mini-transcribe	$0.05–0.15
Full app build (gen + debug rounds)	gpt-5.4	$0.10–0.50
Change build	gpt-5.4	$0.05–0.30

7. Information Flows

Create flow (text)

```
Browser → POST /api/chat {message}
  ■ create conversation (channel=text) on first turn
  ■ runPlanner(history, message)  [Agents SDK]
  ■    ■ propose_spec tool fires once → captured in-process
  ■ persistProposal(): app row (draft) + requirements v1 + approvals(build, pending)
  ■    ■ earlier pending approvals of same type auto-rejected (superseded)
  ■ ← {conversationId, reply, proposal{approvalId,…}}
Browser → POST /api/approvals/:id {decision:approved}
  ■ quota check (monthly, per owner)
  ■ app.status=spec_approved; insert build_runs(queued)
  ■ runInBackground(startBuildPipeline)   [waitUntil on Vercel]
```

Build pipeline (§8 details the state machine)

```
generating: loadTemplate + Code/Change Agent → FileMap
testing:    runner.writeFiles → install→typecheck→lint→test→build
debugging:  on failure ≤5 rounds: Debug Agent rewrites files → re-run from typecheck
            (install-failure reruns from install)
commit:     createRepoIfMissing → createBranch(build-xxxxxxxx) →
            snapshot commit (full tree, no base_tree — no stale files)
deploying:  ensureProject(git-linked, ssoProtection off) →
            createDeployment(gitSource ref=branch) → row(deployments,building) → return
finalize:   GET /status heartbeat → Vercel READY? → smoke test →
            app.previewUrl, run=awaiting_user_test, change_request=complete
```

Publish flow (production)

```
Browser → POST /api/apps/:id/publish
  ■ latest run must be awaiting_user_test
  ■ insert approvals(deploy_production, approved)  ← the button IS the approval
  ■ run=deploying; background: merge build branch → main;
  ■   createDeployment(target=production, ref=main) → deployments(building)
  ■ finalizer: READY → productionUrl (stable alias), app=deployed, run=complete
```

Change flow

The user picks a built app; /api/chat receives appId, verifies ownership, and runs the Change Planner seeded with the latest spec. The proposal bumps the requirement version, records a change_requests row and a pending 'change' approval. On approval the pipeline detects change mode (version > 1 and repo exists), fetches current src files from the repo's default branch, overlays them on a fresh template (so template improvements — e.g. vercel.json — propagate), runs the Change Agent, and continues through the identical test/deploy path. A failed change build never marks a live app failed; production keeps serving.

Voice flow

```
Browser → POST /api/realtime/session {appId?}
  ■ ownership check; change mode ⇒ instructions embed current spec
  ■ conversation row (channel=voice)
  ■ OpenAI POST /v1/realtime/client_secrets (expires ≤15 min) → ek_…
  ■ ← {clientSecret, model, instructions, conversationId}
Browser: RealtimeAgent+RealtimeSession.connect({apiKey: ek}) over WebRTC
  ■ audio.input.transcription: gpt-4o-mini-transcribe (live captions)
  ■ propose tool executes IN BROWSER → POST /api/voice/propose
  ■    ■ server re-validates spec with Zod → persistProposal (same path as text)
  ■ session end/10-min cap → POST /api/voice/transcript
Approve / build / publish: identical to text flow
```

Rollback, delete, heartbeat

Rollback: the app page lists production deployments; 'current' is resolved from Vercel (lastRollbackTarget, falling back to targets.production). Restore tries the Instant Rollback endpoint first, then promote — both directions work (older or newest). No rebuild occurs. **Delete:** removes the Vercel project, the GitHub repo, and all VoiceForge rows in FK-safe order; transcripts and audit rows are kept with references nulled; external failures downgrade to warnings. **Heartbeat:** every GET /status also reaps runs stuck >25 minutes and finalizes pending deployments — the polling UI doubles as the job scheduler.

8. Build Pipeline and Runners

State machine (build_runs.status)

```
queued → generating → testing ■ debugging (≤5 rounds)
       → deploying → awaiting_user_test → (publish) → deploying → complete
any state → failed  [errorMessage set; live apps keep status=deployed]
```

Test gauntlet

Step	Command	Timeout	Recorded as
install	npm install --no-audit --no-fund --ignore-scripts	5 min	suite=build
typecheck	npm run typecheck (tsc --noEmit)	3 min	typecheck
lint	npm run lint (eslint, next/core-web-vitals)	3 min	lint
test	npm run test (vitest run, jsdom)	5 min	unit
build	npm run build (next build)	8 min	build
smoke	HTTPS GET of preview; 200 + HTML required	—	smoke

Step environments deliberately omit NODE_ENV: setting it to 'development' makes next build fail with a misleading prerender error (discovered empirically). Output tails (8 KB) are persisted per attempt; the most recent failure's output is exposed in the UI for diagnosis.

Runner backends

Backend	Where	Mechanics
local	Server not on Vercel (the owner's machine)	child_process.spawn in $TMPDIR/voiceforge-builds/ (override: BUILD_WORKSPACE_DIR); minimal env (PATH, HOME, CI, telemetry off)
sandbox	process.env.VERCEL set (hosted)	@vercel/sandbox microVM (runtime node24, 25-min ceiling); mkdir -p + writeFiles(Buffer); runCommand per step; stopped in finally

Hosted-build caveat: the orchestrating function is subject to Vercel plan limits (300 s on Hobby with Fluid compute, 800 s on Pro). The finalizer removes deployment waits from that budget; long multi-debug builds may still exceed Hobby limits, in which case the stale reaper fails them cleanly and local building remains the fallback.

9. External Integrations

OpenAI

Agents SDK (@openai/agents): Agent/run/tool with Zod v4 parameters for all text agents; maxTurns caps (6 planning, 40 codegen, 25 debug). **Realtime:** POST /v1/realtime/client_secrets mints ek_* ephemeral keys server-side; browsers connect via WebRTC through @openai/agents/realtime (RealtimeAgent/RealtimeSession); input transcription via gpt-4o-mini-transcribe. API key: OPENAI_API_KEY (server only).

GitHub (fine-grained PAT)

Token permissions: Administration R/W (create/delete repos) and Contents R/W (commits), all repositories. Endpoints used via @octokit/rest: repos.get/createForAuthenticatedUser/delete/merge; git.getRef/createRef/updateRef/getCommit/createTree/createCommit/getTree/getBlob. Commits are complete snapshots (tree built without base_tree) so no stale files survive regeneration. One private repo per app; one branch per build; main holds only published code.

Vercel (REST + Sandbox)

Endpoint	Use
POST /v11/projects	Create project git-linked to the app repo (framework=nextjs)
PATCH /v9/projects/:id	Disable ssoProtection so family can open previews
GET /v9/projects/:idOrName	Existence check; current-production detection (lastRollbackTarget ?? targets.production)
DELETE /v9/projects/:name	App deletion
POST /v13/deployments	Create preview (gitSource ref=branch) or production (target=production, ref=main); repoId from GitHub
GET /v13/deployments/:id	Finalizer polling (readyState, url, alias)
POST /v1/projects/:id/rollback/:dpl	Instant Rollback (restore, both directions)
POST /v10/projects/:id/promote/:dpl	Fallback restore path
@vercel/sandbox	Hosted build execution (node24 microVMs)

Clerk and Neon

Clerk: email magic-link sign-in, Restricted (invite-only) sign-up, clerkMiddleware guarding /dashboard and /api; users are mirrored into the users table on first request, with role=admin for ADMIN_EMAILS. Neon: serverless Postgres over the HTTP driver; the Drizzle client is created lazily so the app builds without DATABASE_URL.

10. HTTP API Reference

All routes require a Clerk session; all app-scoped routes verify ownership (admin may read any app's status). Bodies are validated with Zod; errors return {error} with 4xx/5xx.

Route	Method	Purpose / notable responses
/api/chat	POST	{conversationId?, appId?, message ≤2000} → {conversationId, reply, proposal?}; 80-item transcript cap; 502 on planner failure
/api/approvals/:approvalId	POST	{decision: approved|rejected}; 409 if already decided; 429 over quota; queues build (returns buildRunId)
/api/apps/:appId	DELETE	Tri-platform delete → {ok, warnings[]}; 409 while building
/api/apps/:appId/status	GET	{app, buildRun{status,logs,…}, testResults[], failedOutput}; side-effects: reap + finalize
/api/apps/:appId/rebuild	POST	Re-run latest approved spec (build or change); 409 if active; 429 over quota
/api/apps/:appId/publish	POST	Requires awaiting_user_test; records deploy_production approval; async merge+deploy
/api/apps/:appId/rollback	POST	{deploymentId} (VoiceForge id) → restore; 502 with platform message on failure
/api/realtime/session	POST	{appId?} → {clientSecret, model, instructions, conversationId, changeMode}
/api/voice/propose	POST	{conversationId, spec, changeSummary?, transcript[]} → {proposal}; server-side Zod re-validation
/api/voice/transcript	POST	{conversationId, transcript[]} → {ok}

11. Security Model

1. Credential containment. Platform keys never reach the browser or generated apps; the only client-side credential is the ≤15-minute realtime ephemeral key. **2. Real authentication.** Clerk magic-link with invite-only sign-up; display names are cosmetic, identity is the Clerk subject. **3. Ownership enforcement.** Every query is scoped to the owner (admin read-all where explicitly intended). **4. Double approval gates.** Builds require an approved build/change approval row; production requires a deploy_production approval; approvals of superseded specs are auto-rejected. **5. Constrained codegen.** Path policy (src-only, .ts/.tsx, protected files), pinned dependencies, script-less installs, sandboxed or workspace-isolated execution. **6. Untrusted-code posture.** Generated apps run only in the test runner and on Vercel's infrastructure, never inside the VoiceForge process; hosted mode uses microVM isolation. **7. Audit trail.** Every credentialed action writes audit_logs; deletion preserves history. **8. Cost controls.** Monthly per-user build quotas, capped planning transcripts, 10-minute voice sessions, single-flight builds per app, stale-run reaping, duplicate-deploy prevention. **9. Input validation.** Zod on every route body and every agent tool payload (voice proposals re-validated server-side). **10. Blast-radius limits.** Per-app repos/projects mean a compromised or broken app never touches another; rollback provides instant recovery; a failed change cannot take down the live version.

12. Environment Variables

Variable	Required	Purpose
NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	yes	Clerk browser key (public by design)
CLERK_SECRET_KEY	yes	Clerk server key
NEXT_PUBLIC_CLERK_SIGN_IN_URL	yes	/sign-in
DATABASE_URL	yes	Neon pooled connection string
ADMIN_EMAILS	yes	Comma-separated admin allowlist (role at first sign-in)
OPENAI_API_KEY	yes	All agents + realtime secret minting
OPENAI_PLANNER_MODEL	no (gpt-5.4-mini)	Planner/Change Planner
OPENAI_CODER_MODEL	no (gpt-5.4)	Code/Change/Debug agents
OPENAI_REALTIME_MODEL	no (gpt-realtime-2.1-mini)	Voice sessions
GITHUB_TOKEN	yes (builds)	Fine-grained PAT: Administration R/W, Contents R/W
GITHUB_OWNER	yes (builds)	Account owning voiceforge-* repos
VERCEL_TOKEN	yes (deploys)	Vercel REST bearer token
VERCEL_TEAM_ID	no	Only if projects live in a team
BUILD_WORKSPACE_DIR	no	Local build workspace override

13. Operational Characteristics

Build duration: 3–8 minutes typical (npm install of the generated app dominates; each debug round adds ~30–60 s of model time plus re-testing). **Concurrency:** one active build per app enforced at queue time. **Quota accounting:** build_runs joined to app ownership, calendar-month window. **Monitoring:** the admin page surfaces usage counts, failure counts, recent runs with errors, and the audit timeline; build pages expose per-step outputs. **Recovery:** failed builds offer one-click retry (fresh generation under current rules); stuck runs are auto-failed at 25 minutes; production issues are reversible via instant rollback. **Data retention:** transcripts and audit logs survive app deletion.

14. Known Limitations and Roadmap

Current limitations. Generated apps are client-side only: no server persistence, no multi-user shared data, no authentication, and no AI calls inside generated apps (the planner records aiFeatures but the template forbids external services). Pipeline test suites cover typecheck/lint/unit/build/smoke; Playwright e2e, accessibility, and dependency-security scans from the spec are not yet wired. Hosted builds are constrained by Vercel function limits (Hobby: 300 s). Voice sessions cap at 10 minutes. Rollback relies on partially documented Vercel endpoints (with a documented fallback).

Stage 7 candidates. (1) AI-enabled generated apps: a locked /api/ai route in the template with per-app OPENAI_API_KEY env vars set at deploy time, pinned cheap models, and per-app daily request limits. (2) A Mac-resident build worker that claims queued build_runs from the database so family builds run on the owner's always-on machine while everyone uses the hosted UI. (3) Server-side data for generated apps (shared family lists) via per-app Neon schemas and a template data-access layer. (4) Playwright + accessibility + dependency scanning in the gauntlet. (5) Quota self-service and email notifications (build finished, publish reminders).